Sistema de gestión de carreteras

Proyecto completado: formularios, controladores y consultas

C# • Windows Forms • .NET Framework 4.7.2 • Varias listas de objetos • LINQ

Esta entrega parte de SistemaCarreteras.zip y conserva la organización por Entidades, Controladores y Formularios. Incluye la solución completa, instrucciones de uso y el código de los formularios para estudiarlo o copiarlo.

Qué se entrega

1. SistemaCarreteras_Completo.zip: solución y fuentes completas, diseños, recursos y pruebas. 2. Este documento en Word y PDF: explicación y anexos de código. Los anexos contienen los archivos .cs de los formularios, el Designer nuevo de consultas, los controladores y Program.cs. Los Designer originales y las entidades están en el ZIP.

Resultado de la verificación

85 comprobaciones aprobadas en Linux con Mono y pantalla virtual. Se abrieron los 14 formularios, se ejecutaron las cinco consultas y se comprobaron duplicados, rangos y restricciones. La compilación de las fuentes no emitió errores ni advertencias; la compilación del proyecto completo con XBuild terminó sin errores y con una advertencia del toolset sobre .NET Framework 4.7.2. No se ejecutó Visual Studio en Windows; esa revisión final queda pendiente en el equipo de entrega.

Lectura recomendada

Primero lee las secciones 1 a 7. Después consulta el anexo A para los formularios y el anexo B para los controladores y el inicio del programa. Para ejecutar, usa el ZIP completo: no basta con copiar únicamente los archivos de lógica sobre otra versión del diseño.

1. Relación con los ejercicios de clase

Se revisaron los proyectos de referencia enviados. Gimnasio y ParqueAtracciones muestran el patrón de formulario modal con CrearObjeto, ValidarDatos, Guardar y setDatos; Banco también conserva el objeto de cada fila en Tag. Se mantuvieron esos patrones, sin introducir repositorios genéricos, servicios, una base de datos ni otra arquitectura.

Capa	Responsabilidad
Entidades	Categoria, Carretera, Comuna, Tramo, ComunaRecorrida y ExtremoCarretera: representan los datos y relaciones. Se conservan sus atributos privados, constructores y propiedades.
Controladores	Las clases TLista contienen listas estáticas y operaciones de agregar, buscar, modificar, eliminar y validar. TListaDatos centraliza los ejemplos iniciales.
Formularios	Los frm de captura validan y crean objetos. Los frmAdmin muestran, buscan y realizan las operaciones permitidas. frmPrincipal abre los módulos; frmConsultas muestra las consultas LINQ.

1.1. Secuencia de un registro

El administrador abre el formulario con ShowDialog. Guardar valida los controles y asigna DialogResult.OK. Al recibir OK, el administrador llama a CrearObjeto y después al controlador. Solo entonces actualiza el listado. Cancelar cierra el diálogo sin cambiar la lista.

Para modificar, setDatos copia los valores del objeto a los controles. No se escribe sobre el objeto original durante la edición. La fila conserva su referencia en Tag; así una búsqueda no cambia la identidad del registro seleccionado.

1.2. Ajustes necesarios

Se corrigió publiuc por public en TListaTramo. Program.cs ahora inicia frmPrincipal; se quitó el Designer sobrante de Form1 que no formaba parte del proyecto. Se enlazó el menú Carreteras y se completó Consultas. Se corrigieron rótulos copiados de otros formularios y textBox2 pasó a llamarse txtObservacion en frmExtremo.

Se conservan las entidades. En los controladores se reforzaron las modificaciones y la integridad de las relaciones. Categoría, carretera, comuna y tramo se actualizan conservando el objeto al que apuntan las demás listas. La modificación de recorridos admite cambiar la clave compuesta sin eliminar primero el registro.

2. Abrir el proyecto en Visual Studio

1. Descomprime el ZIP en una carpeta local.

2. Abre SistemaCarreteras.sln en Visual Studio 2022. Se incluye la solución tradicional para evitar depender del soporte de .slnx.

3. Comprueba la carga de trabajo Desarrollo de escritorio de .NET y el Targeting Pack/Developer Pack de .NET Framework 4.7.2. No cambies el framework por .NET moderno.

4. Si es necesario, selecciona SistemaCarreteras como proyecto de inicio. Compila la solución y pulsa F5.

5. El menú principal permite entrar a categorías, carreteras, comunas, tramos, recorridos, extremos y consultas.

No se necesitan paquetes NuGet, SQL Server, usuario de base de datos ni claves. Las listas se inicializan una sola vez por ejecución mediante TListaDatos.CargarDatos, aunque se abran y cierren módulos repetidamente.

2.1. Datos de práctica

Lista	Registros iniciales
Categorías	5: Local, Comercial, Regional, Nacional y Autovía.
Carreteras	5, códigos 101 a 105. Cada una tiene dos tramos de 10 km.
Comunas	6. La comuna 6 no tiene relaciones y permite probar una eliminación válida.
Tramos	10: números 1 y 2 por carretera; rangos 0–10 y 10–20.
Recorridos	15: dos comunas en el primer tramo y una en el segundo de cada carretera.
Extremos	6: cinco inicios físicos y un final de confluencia para carretera 101 hacia carretera 102.

Los nombres geográficos y las uniones son ejemplos académicos; no representan la red vial real. Se dejan extremos finales sin registrar para practicar las altas. El sistema permite como máximo un inicio y un final por carretera, sin crear automáticamente los que falten.

2.2. Duración de los datos

Las listas existen en memoria. Lo añadido, modificado o eliminado permanece mientras la aplicación está abierta. Al cerrar y volver a abrir, se vuelven a cargar los ejemplos. No se agregó guardado en archivos ni base de datos, porque el planteamiento y los proyectos trabajan con listas de objetos.

3. Funciones de cada módulo

Módulo	Operaciones disponibles según el PDF
Categorías	Registrar y listar; valida código positivo, nombre seleccionado y código no repetido.
Carreteras	Listar y buscar por código, nombre o categoría. Muestra la longitud total sumando sus tramos.
Comunas	Listar, buscar por nombre/cantón/provincia y eliminar solo si no hay recorridos o extremos relacionados.
Tramos	Consultar, ordenar por número y buscar por carretera, número o rango de kilómetros. Muestra la longitud calculada.
Comunas recorridas	Insertar, modificar, eliminar y buscar. Se puede cambiar carretera, tramo, comuna, orden o kilómetros si el resultado respeta las relaciones.
Extremos	Insertar, modificar, eliminar y buscar. Carretera y tipo de extremo quedan fijos en edición; el resto de los datos puede modificarse si es válido.
Consultas	Cinco consultas LINQ con filtros seleccionables y total de resultados.

Los formularios de captura frmCarretera, frmComuna y frmTramo también tienen código completo y se pueden reutilizar con ShowDialog, CrearObjeto y el controlador correspondiente. No se agregaron altas a sus administradores: el PDF pide lectura de carreteras/tramos y eliminación/lectura de comunas. Las pruebas usan esos formularios y comprueban que abren correctamente.

3.1. Búsquedas y selección

Selecciona el criterio y escribe el texto; el listado se actualiza. Mostrar todas limpia el filtro. Las búsquedas textuales no distinguen mayúsculas. Para tramos, carretera acepta nombre o código; el número se compara exactamente.

En Rango de kilómetros escribe desde;hasta, por ejemplo 2;8. Se muestran los intervalos que intersectan el rango, incluidos sus límites. El punto y coma separa ambos valores; los decimales siguen la configuración regional de Windows. Un rango mal escrito vacía el resultado y muestra una indicación de formato.

Para eliminar o modificar selecciona una fila. No se toma su posición como índice de la lista original: se utiliza el objeto guardado en Tag. La eliminación requiere confirmar Sí y se rechaza si tiene relaciones que deben conservarse.

4. Validaciones y reglas de negocio

Objeto	Reglas aplicadas
Identificadores	Código o número entero positivo; códigos de catálogo únicos y número de tramo único dentro de cada carretera.
Tramos	Carretera existente, km inicial no negativo, km final mayor que inicial y valores finitos. Longitud = km final − km inicial. La carretera de un tramo queda bloqueada al editar.
Recorridos	Tramo y comuna existentes; orden positivo y no repetido en el mismo tramo; entrada ≤ salida; kilómetros dentro del tramo. Una misma comuna puede estar en varios tramos.
Extremos	Carretera, tramo y comuna válidos; tramo de la carretera indicada; máximo un inicio y un final. El inicio coincide con el menor km inicial y el final con el mayor km final de esa carretera.
Confluencias	Destino distinto del origen; carretera, tramo, comuna, km y observación obligatorios. El tramo pertenece al destino y el km pertenece tanto al tramo como al recorrido de la comuna indicada.
Final físico	No obliga destino. Al seleccionarlo, el formulario desactiva y limpia los controles de confluencia y crea el objeto sin referencias de destino.
Integridad	Impide eliminar categorías con carreteras; carreteras con tramos/extremos; tramos con recorridos/extremos; comunas usadas por recorridos/extremos.

4.1. Protección adicional de relaciones

La comuna de un extremo debe estar registrada como comuna recorrida de su tramo y cubrir el kilómetro indicado. Un recorrido que sostiene un extremo no se puede eliminar o reducir si deja ese punto sin cobertura; primero se modifica o elimina el extremo. Igualmente, ampliar o reducir tramos no puede invalidar los extremos ya registrados.

Estas comprobaciones mantienen coherentes las relaciones al editar. Los controles desplegables solo usan objetos de las listas existentes. Los controladores siguen devolviendo bool para indicar éxito o rechazo, como en el proyecto original.

4.2. Claves al modificar

Carretera se identifica por Codigo; tramo por carretera y Numero; recorrido por carretera, número de tramo y Orden; extremo por carretera y TipoExtremo. Para recorridos se conserva el original durante la edición y se revisa si la nueva clave está libre antes de reemplazarlo. Un rechazo deja intacto el registro anterior.

5. Consultas LINQ

El PDF recibido usa el título LINQ to SQL, pero exige varias listas de objetos y no incluye base de datos ni modelo SQL. Esta solución usa LINQ to Objects sobre las listas. No se presenta como implementación de LINQ to SQL.

El archivo pide al menos cinco consultas, pero únicamente detalla las consultas 1 y 2. Se implementan esas dos y se añaden las consultas 3, 4 y 5 como complementarias. Si hay otra parte del enunciado, deberán cotejarse con ella.

Consulta	Filtro y resultado
1. Carreteras por categoría	Categoría seleccionada. Código, nombre, categoría y longitud total de las carreteras de esa categoría. Usa Where, Select y Sum.
2. Tramos de una carretera	Carretera seleccionada. Número de tramo, km inicial, km final y longitud, ordenados por número con OrderBy.
3. Comunas recorridas por tramo (complementaria)	Carretera y tramo seleccionados. Comuna, orden, km entrada y km salida, ordenados por recorrido.
4. Extremos de una carretera (complementaria)	Carretera seleccionada. Inicio/final, tramo, comuna, km, tipo de finalización y carretera de destino.
5. Confluencias entre carreteras (complementaria)	Todas las confluencias: origen, extremo, tramo/comuna/km origen, carretera/tramo/comuna/km destino y observación.

En Consultas selecciona el tipo de consulta; se habilitan solamente los filtros que utiliza. Pulsa Consultar para actualizar. Al cambiar un filtro se limpia el resultado anterior, para no mostrar datos que correspondan a otra selección.

Ejemplo esperado

En consulta 1, categoría Regional, aparece carretera 101 con longitud total 20. En consulta 2, carretera 101, aparecen los tramos 1 y 2. La consulta 5 muestra la confluencia de carretera 101 a carretera 102, con kilómetro destino 7 y su observación.

6. Pruebas y revisión antes de entregar

Se compilaron las fuentes de C# sin errores ni advertencias. También se compiló el .csproj completo, incluidos los recursos, usando Mono/XBuild: cero errores y una advertencia de compatibilidad del toolset con v4.7.2. Las 85 comprobaciones se ejecutaron sobre ese ensamblado. XBuild no sustituye a una prueba final de Visual Studio y Windows.

Grupo	Qué se comprobó
Carga inicial	Cantidades esperadas y que volver a cargar no duplique ni reponga registros eliminados.
Formularios	Los 14 formularios abren y crean su ventana. Se probaron cascadas de carretera/tramo, setDatos y retorno de Guardar.
Listados y consultas	Filtros de carretera/categoría/cantón/rango; referencia correcta de la fila filtrada; cinco consultas y suma de longitudes.
Validaciones	Duplicados, orden, longitud cero, km negativos, NaN e infinito; recorridos y confluencias válidos e inválidos.
Modificación y eliminación	Cambio de orden sin pisar otro registro, conservación de referencias y rechazo de cambios/eliminaciones que dañen relaciones.

Los registros de salida están en Pruebas/Resultados_Mono.txt y Pruebas/Compilacion_Mono.txt. El código de verificación está en Pruebas/Pruebas.cs y no forma parte del ejecutable de la aplicación.

6.1. Repetir en Windows

Abre Developer PowerShell para Visual Studio, entra a la carpeta Pruebas y ejecuta .\Ejecutar-Pruebas.ps1. El script compila la solución, compila las comprobaciones con el compilador de .NET Framework y las ejecuta. Se entrega para facilitar la revisión; no se ha ejecutado en Windows en este entorno.

6.2. Recorrido manual recomendado

1. Ejecuta F5 y abre cada opción del menú. Verifica que Regresar retorna a la principal.

2. En Categorías inserta un código nuevo; comprueba el listado. Intenta reutilizar un código para ver el aviso.

3. En Carreteras busca 101 y después Regional por categoría. En Tramos prueba el rango 2;8 y un rango mal escrito.

4. En Comunas elimina la comuna 6. Intenta eliminar Machala para comprobar el bloqueo por relaciones.

5. En Recorridos modifica carretera 101, tramo 1, orden 2 a orden 3. Prueba Cancelar y verifica que no cambie. Prueba también un orden ya ocupado.

6. En Extremos crea un final físico para carretera 102, tramo 2, comuna Santa Rosa, km 20. Para una confluencia, rellena todos los datos de destino y observación.

7. Ejecuta las cinco consultas y revisa los filtros. Cierra la aplicación y vuelve a abrir: los cambios de la sesión se descartan y se cargan los ejemplos.

7. Archivos y métodos para estudiar

Archivo	Métodos principales
frmPrincipal.cs	AbrirFormulario y eventos de las siete opciones del menú.
frmCategoria.cs / frmadminCategoria.cs	CargarDatos, CrearObjeto, ValidarDatos y Guardar / Nuevo y Listar.
frmCarretera.cs / frmComuna.cs	CrearObjeto, ValidarDatos, setDatos y Guardar; capturas reutilizables.
frmTramo.cs	CrearObjeto, ValidarDatos, setDatos y CalcularLongitud.
frmComunaRecorrida.cs	CargarTramos, CrearObjeto, ValidarDatos, setDatos y Guardar.
frmExtremo.cs	CargarTramos, CargarTramosDestino, HabilitarDestino, CrearObjeto y ValidarDatos.
frmAdminCarretera / Comuna / Tramo	CargarDatos, Listar y Buscar; Comuna añade Seleccionado y Eliminar.
frmAdminComunaRecorrida / Extremo	Nuevo, Actualizar, Eliminar y Seleccionado; objetos en Tag.
frmConsultas.cs	CargarDatos, CargarTramos, HabilitarFiltros y Consultar.
TListaComunaRecorrida.cs	Validar, PuedeModificar, Modificar(original, nuevo), ContienePunto y protección de extremos.
TListaDatos.cs	Carga de ejemplos una sola vez por ejecución.

Los eventos que ya estaban enlazados en un Designer se mantienen allí. En los formularios que no tenían eventos, los enlaces se añaden después de InitializeComponent en el constructor, evitando registrar dos veces el mismo manejador. frmConsultas tiene su propio Designer nuevo.

Si sustituyes archivos en una copia anterior, necesitas también los cambios en el Designer de frmExtremo (txtObservacion), el nuevo formulario de consultas y sus entradas en el .csproj. La forma más sencilla de evitar desajustes es abrir directamente la solución del ZIP completo.

Anexos de código

Anexo A: todos los formularios de lógica y el Designer nuevo de consultas. Anexo B: los siete controladores y el punto de entrada. El ZIP conserva el resto de diseños, recursos, entidades y archivos de solución. El código de los anexos corresponde a la versión entregada; las líneas largas pueden ajustarse visualmente al ancho del documento.

Anexo A.1 — frmPrincipal

Archivo: Formularios/frmPrincipal.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmPrincipal : Form
    {
        public frmPrincipal()
        {
            InitializeComponent();
            carreterasToolStripMenuItem.Click += carreterasToolStripMenuItem_Click;
        }
        public void AbrirFormulario(Form formulario)
        {
            using (formulario) formulario.ShowDialog();
            lblEstado.Text = "Sistema listo";
        }
        private void frmPrincipal_Load(object sender, EventArgs e)
        {
            SistemaCarreteras.Controladores.TListaDatos.CargarDatos();
        }
        private void categoríasToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Administración de categorías";
            frmadminCategoria frm = new frmadminCategoria();
            AbrirFormulario(frm);
        }
        private void comunasToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Administración de comunas";
            frmAdminComuna frm = new frmAdminComuna();
            AbrirFormulario(frm);
        }
        private void tramosToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Administración de tramos";
            frmAdminTramo frm = new frmAdminTramo();
            AbrirFormulario(frm);
        }
        private void comunasRecorridasToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Administración de comunas recorridas";
            frmAdminComunaRecorrida frm = new frmAdminComunaRecorrida();
            AbrirFormulario(frm);
        }
        private void extremosDeCarreteraToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Administración de extremos";
            frmAdminExtremo frm = new frmAdminExtremo();
            AbrirFormulario(frm);
        }
        private void consultasToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Consultas LINQ";
            AbrirFormulario(new frmConsultas());
        }
        private void carreterasToolStripMenuItem_Click(object sender, EventArgs e)
        {
            lblEstado.Text = "Consulta de carreteras";
            AbrirFormulario(new frmAdminCarretera());
        }
    }
}
```

Anexo A.2 — frmCategoria

Archivo: Formularios/frmCategoria.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmCategoria : Form
    {
        public frmCategoria()
        {
            InitializeComponent();
        }
        public void CargarDatos()
        {
            cboNombre.Items.Clear();
            cboNombre.Items.Add("Local");
            cboNombre.Items.Add("Comercial");
            cboNombre.Items.Add("Regional");
            cboNombre.Items.Add("Nacional");
            cboNombre.Items.Add("Autovía");
            cboNombre.DropDownStyle = ComboBoxStyle.DropDownList;
        }
        public Categoria CrearObjeto()
        {
            int codigo = Convert.ToInt32(txtCodigo.Text);
            string nombre = cboNombre.Text;
            Categoria resultado = new Categoria(codigo, nombre);
            return resultado;
        }
        public bool ValidarDatos()
        {
            bool resultado = true;
            int codigo;
            if (txtCodigo.Text.Trim() == "")
            {
                MessageBox.Show("Ingrese el código de la categoría.");
                resultado = false;
            }
            else if (!int.TryParse(txtCodigo.Text, out codigo))
            {
                MessageBox.Show("El código debe ser un número entero.");
                resultado = false;
            }
            else if (codigo <= 0)
            {
                MessageBox.Show("El código debe ser mayor que cero.");
                resultado = false;
            }
            else if (cboNombre.SelectedIndex == -1)
            {
                MessageBox.Show("Seleccione una categoría.");
                resultado = false;
            }
            else if (TListaCategoria.Buscar(codigo) != null)
            {
                MessageBox.Show("Ya existe una categoría con ese código.");
                resultado = false;
            }
            return resultado;
        }
        public void Guardar()
        {
            if (ValidarDatos()) this.DialogResult = DialogResult.OK;
        }
        private void frmCategoria_Load(object sender, EventArgs e)
        {
            CargarDatos();
        }
        private void btnCancelar_Click(object sender, EventArgs e)
        {
            this.Close();
        }
        private void btnGuardar_Click(object sender, EventArgs e)
        {
            Guardar();
        }
    }
```

```
}
```

Anexo A.3 — frmadminCategoria

Archivo: Formularios/frmadminCategoria.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmadminCategoria : Form
    {
        public frmadminCategoria()
        {
            InitializeComponent();
        }
        public void CargarDatos() { TListaDatos.CargarDatos(); }
        public void Listar()
        {
            dgvCategorias.DataSource = null;
            dgvCategorias.DataSource = TListaCategoria.GetLista();
            lblTotal.Text = "Total de categorías: " +
                TListaCategoria.listaCategorias.Count;
        }
        public void Nuevo()
        {
            using (frmCategoria frm = new frmCategoria())
            {
                if (frm.ShowDialog() == DialogResult.OK)
                {
                    bool resultado = TListaCategoria.Agregar(frm.CrearObjeto());
                    MessageBox.Show(resultado ? "Categoría registrada correctamente." : "No se pudo registrar la
categoría.");
                    Listar();
                }
            }
        }
        private void frmadminCategoria_Load(object sender, EventArgs e)
        {
            CargarDatos();
            Listar();
            dgvCategorias.ReadOnly = true;
            dgvCategorias.SelectionMode =
                DataGridViewSelectionMode.FullRowSelect;
            dgvCategorias.MultiSelect = false;
            dgvCategorias.AllowUserToAddRows = false;
            dgvCategorias.AutoSizeColumnsMode =
                DataGridViewAutoSizeColumnsMode.Fill;
        }
        private void btnInsertar_Click(object sender, EventArgs e)
        {
            Nuevo();
        }
        private void btnRegresar_Click(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

Anexo A.4 — frmAdminCarretera

Archivo: Formularios/frmAdminCarretera.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmAdminCarretera : Form
    {
        public frmAdminCarretera()
        {
            InitializeComponent();
        }
        public void CargarDatos() { TListaDatos.CargarDatos(); }
        public void Listar()
        {
            var datos = TListaCarretera.GetLista()
                .Select(x => new
                {
                    x.Codigo,
                    x.Nombre,
                    Categoria = x.NombreCategoria,
                    LongitudTotal = TListaTramo.GetLista().Where(t => t.Carretera.Codigo == x.Codigo).Sum(t =>
t.Longitud)
                }).ToList();
            dgvCarreteras.DataSource = null;
            dgvCarreteras.DataSource = datos;
            lblTotal.Text = "Total de carreteras: " + datos.Count;
        }
        public void Buscar()
        {
            string texto = txtBuscar.Text.Trim().ToLower();
            string criterio = cboCategoria.Text;
            var consulta = TListaCarretera.GetLista().AsEnumerable();
            if (texto != "")
            {
                if (criterio == "Código")
                {
                    consulta = consulta.Where(x =>
                        x.Codigo.ToString().Contains(texto));
                }
                else if (criterio == "Nombre")
                {
                    consulta = consulta.Where(x =>
                        x.Nombre != null &&
                        x.Nombre.ToLower().Contains(texto));
                }
                else if (criterio == "Categoría")
                {
                    consulta = consulta.Where(x =>
                        x.NombreCategoria.ToLower().Contains(texto));
                }
            }
            var datos = consulta.Select(x => new
            {
                x.Codigo,
                x.Nombre,
                Categoria = x.NombreCategoria,
                    LongitudTotal = TListaTramo.GetLista().Where(t => t.Carretera.Codigo == x.Codigo).Sum(t =>
t.Longitud)
            }).ToList();
            dgvCarreteras.DataSource = null;
            dgvCarreteras.DataSource = datos;
            lblTotal.Text = "Total de carreteras: " + datos.Count;
        }
        private void frmAdminCarretera_Load(object sender, EventArgs e)
        {
            cboCategoria.Items.Clear();
            cboCategoria.Items.Add("Código");
            cboCategoria.Items.Add("Nombre");
            cboCategoria.Items.Add("Categoría");
            cboCategoria.DropDownStyle = ComboBoxStyle.DropDownList;
            cboCategoria.SelectedIndex = 0;
            dgvCarreteras.ReadOnly = true;
            dgvCarreteras.SelectionMode =
                DataGridViewSelectionMode.FullRowSelect;
            dgvCarreteras.MultiSelect = false;
            dgvCarreteras.AllowUserToAddRows = false;
```

```
            dgvCarreteras.AutoSizeColumnsMode =
                DataGridViewAutoSizeColumnsMode.Fill;
            CargarDatos();
            Listar();
        }
        private void btnRegresar_Click(object sender, EventArgs e)
        {
            this.Close();
        }
        private void cboCategoria_SelectedIndexChanged(object sender, EventArgs e)
        {
            Buscar();
        }
        private void btnMostrarTodas_Click(object sender, EventArgs e)
        {
            txtBuscar.Clear();
            cboCategoria.SelectedIndex = 0;
            Listar();
        }
        private void txtBuscar_TextChanged(object sender, EventArgs e)
        {
            Buscar();
        }
    }
}
```

Anexo A.5 — frmCarretera

Archivo: Formularios/frmCarretera.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmCarretera : Form
    {
        public frmCarretera()
        {
            InitializeComponent();
            CargarDatos();
            btnGuardar.Click += btnGuardar_Click;
            btnCancelar.Click += btnCancelar_Click;
            AcceptButton = btnGuardar;
            CancelButton = btnCancelar;
        }
        private bool modificando;
        public void CargarDatos()
        {
            cboNombre.Items.Clear();
            cboNombre.DropDownStyle = ComboBoxStyle.DropDownList;
            foreach (Categoria op in TListaCategoria.GetLista()) cboNombre.Items.Add(op);
        }
        public Carretera CrearObjeto()
        {
            return new Carretera(int.Parse(txtCodigo.Text), txtNombre.Text.Trim(),
(Categoria)cboNombre.SelectedItem);
        }
        private bool Avisar(string mensaje)
        {
            MessageBox.Show(mensaje, "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
        public bool ValidarDatos()
        {
            int codigo;
            if (!int.TryParse(txtCodigo.Text, out codigo) || codigo <= 0)
                return Avisar("Ingrese un código entero mayor que cero.");
            if (string.IsNullOrWhiteSpace(txtNombre.Text) || cboNombre.SelectedItem == null)
                return Avisar("Ingrese el nombre y seleccione la categoría.");
            if (!modificando && TListaCarretera.Buscar(codigo) != null)
                return Avisar("Ya existe una carretera con ese código.");
            return true;
        }
        public void setDatos(Carretera op)
        {
            modificando = true;
            txtCodigo.Text = op.Codigo.ToString();
            txtCodigo.ReadOnly = true;
            txtNombre.Text = op.Nombre;
            cboNombre.SelectedItem = op.Categoria;
        }
        public void Guardar()
        {
            if (ValidarDatos()) DialogResult = DialogResult.OK;
        }
        private void btnGuardar_Click(object sender, EventArgs e) { Guardar(); }
        private void btnCancelar_Click(object sender, EventArgs e) { DialogResult = DialogResult.Cancel; }
    }
}
```

Anexo A.6 — frmAdminComuna

Archivo: Formularios/frmAdminComuna.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmAdminComuna : Form
    {
        public frmAdminComuna()
        {
            InitializeComponent();
            Load += Formulario_Load;
            btnRegresar.Click += btnRegresar_Click;
            btnMostrarTodas.Click += btnMostrarTodas_Click;
            txtBuscar.TextChanged += Filtro_Cambiado;
            cboBuscarPor.SelectedIndexChanged += Filtro_Cambiado;
            Button btnEliminar = new Button();
            btnEliminar.Text = "Eliminar";
            btnEliminar.SetBounds(btnMostrarTodas.Left - 125, btnMostrarTodas.Top, 110, btnMostrarTodas.Height);
            btnEliminar.Click += btnEliminar_Click;
            Controls.Add(btnEliminar);
        }
        public void CargarDatos()
        {
            TListaDatos.CargarDatos();
            cboBuscarPor.Items.Clear();
            cboBuscarPor.Items.AddRange(new object[] { "Nombre", "Cantón", "Provincia" });
            cboBuscarPor.DropDownStyle = ComboBoxStyle.DropDownList;
            cboBuscarPor.SelectedIndex = 0;
            dgvComunas.DataSource = null;
            dgvComunas.Columns.Clear();
            dgvComunas.AutoGenerateColumns = false;
            dgvComunas.ReadOnly = true;
            dgvComunas.AllowUserToAddRows = false;
            dgvComunas.AllowUserToDeleteRows = false;
            dgvComunas.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvComunas.MultiSelect = false;
            dgvComunas.Columns.Add("col0", "Código");
            dgvComunas.Columns.Add("col1", "Nombre");
            dgvComunas.Columns.Add("col2", "Cantón");
            dgvComunas.Columns.Add("col3", "Provincia");
        }
        public void Listar() { Buscar(); }
        public void Buscar()
        {
            dgvComunas.Rows.Clear();
            string texto = txtBuscar.Text.Trim().ToLower();
            string criterio = cboBuscarPor.Text;
            var datos = TListaComuna.GetLista().Where(op => texto == "" ||
                    (criterio == "Nombre" && op.Nombre.ToLower().Contains(texto)) ||
                    (criterio == "Cantón" && op.Canton.ToLower().Contains(texto)) ||
                    (criterio == "Provincia" && op.Provincia.ToLower().Contains(texto)));
            foreach (Comuna op in datos)
            {
                int fila = dgvComunas.Rows.Add(op.Codigo, op.Nombre, op.Canton, op.Provincia);
                dgvComunas.Rows[fila].Tag = op;
            }
            lblTotal.Text = "Total de comunas: " + dgvComunas.Rows.Count;
            dgvComunas.ClearSelection();
        }
        private Comuna Seleccionado()
        {
            if (dgvComunas.SelectedRows.Count == 0) return null;
            return dgvComunas.SelectedRows[0].Tag as Comuna;
        }
        public void Eliminar()
        {
            Comuna op = Seleccionado();
            if (op == null) { MessageBox.Show("Seleccione una fila para eliminar."); return; }
            if (MessageBox.Show("¿Desea eliminar el registro seleccionado?", "Eliminar", MessageBoxButtons.YesNo,
                MessageBoxIcon.Question) == DialogResult.Yes)
            {
                bool resultado = TListaComuna.Eliminar(op.Codigo);
                MessageBox.Show(resultado ? "Registro eliminado correctamente." : "No se puede eliminar: tiene
relaciones con tramos o extremos.");
                Listar();
            }
        }
        private void btnEliminar_Click(object sender, EventArgs e) { Eliminar(); }
        private void Formulario_Load(object sender, EventArgs e) { CargarDatos(); Listar(); }
        private void Filtro_Cambiado(object sender, EventArgs e) { if (dgvComunas.Columns.Count > 0) Buscar(); }
        private void btnMostrarTodas_Click(object sender, EventArgs e) { txtBuscar.Clear();
```

```
cboBuscarPor.SelectedIndex = 0; Listar(); }
        private void btnRegresar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

Anexo A.7 — frmComuna

Archivo: Formularios/frmComuna.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmComuna : Form
    {
        public frmComuna()
        {
            InitializeComponent();
            label1.Text = "DATOS DE LA COMUNA";
            CargarDatos();
            btnGuardar.Click += btnGuardar_Click;
            btnCancelar.Click += btnCancelar_Click;
            AcceptButton = btnGuardar;
            CancelButton = btnCancelar;
        }
        private bool modificando;
        public void CargarDatos()
        {
        }
        public Comuna CrearObjeto()
        {
            return new Comuna(int.Parse(txtCodigo.Text), txtNombre.Text.Trim(), txtCanton.Text.Trim(),
txtProvincia.Text.Trim());
        }
        private bool Avisar(string mensaje)
        {
            MessageBox.Show(mensaje, "Validación", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }
        public bool ValidarDatos()
        {
            int codigo;
            if (!int.TryParse(txtCodigo.Text, out codigo) || codigo <= 0)
                return Avisar("Ingrese un código entero mayor que cero.");
            if (string.IsNullOrWhiteSpace(txtNombre.Text) || string.IsNullOrWhiteSpace(txtCanton.Text) ||
string.IsNullOrWhiteSpace(txtProvincia.Text))
                return Avisar("Complete nombre, cantón y provincia.");
            if (!modificando && TListaComuna.Buscar(codigo) != null)
                return Avisar("Ya existe una comuna con ese código.");
            return true;
        }
        public void setDatos(Comuna op)
        {
            modificando = true;
            txtCodigo.Text = op.Codigo.ToString();
            txtCodigo.ReadOnly = true;
            txtNombre.Text = op.Nombre;
            txtCanton.Text = op.Canton;
            txtProvincia.Text = op.Provincia;
        }
        public void Guardar()
        {
            if (ValidarDatos()) DialogResult = DialogResult.OK;
        }
        private void btnGuardar_Click(object sender, EventArgs e) { Guardar(); }
        private void btnCancelar_Click(object sender, EventArgs e) { DialogResult = DialogResult.Cancel; }
    }
}
```

Anexo A.8 — frmAdminTramo

Archivo: Formularios/frmAdminTramo.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmAdminTramo : Form
    {
        public frmAdminTramo()
        {
            InitializeComponent();
            Load += Formulario_Load;
            btnRegresar.Click += btnRegresar_Click;
            btnMostrarTodas.Click += btnMostrarTodas_Click;
            txtBuscar.TextChanged += Filtro_Cambiado;
            cboBuscarPor.SelectedIndexChanged += Filtro_Cambiado;
            txtBuscar.Width = Math.Min(txtBuscar.Width, cboBuscarPor.Left - txtBuscar.Left - 15);
        }
        public void CargarDatos()
        {
            TListaDatos.CargarDatos();
            cboBuscarPor.Items.Clear();
            cboBuscarPor.Items.AddRange(new object[] { "Carretera", "Número", "Rango de kilómetros" });
            cboBuscarPor.DropDownStyle = ComboBoxStyle.DropDownList;
            cboBuscarPor.SelectedIndex = 0;
            dgvTramos.DataSource = null;
            dgvTramos.Columns.Clear();
            dgvTramos.AutoGenerateColumns = false;
            dgvTramos.ReadOnly = true;
            dgvTramos.AllowUserToAddRows = false;
            dgvTramos.AllowUserToDeleteRows = false;
            dgvTramos.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvTramos.MultiSelect = false;
            dgvTramos.Columns.Add("col0", "Código carretera");
            dgvTramos.Columns.Add("col1", "Carretera");
            dgvTramos.Columns.Add("col2", "Número");
            dgvTramos.Columns.Add("col3", "Km inicial");
            dgvTramos.Columns.Add("col4", "Km final");
            dgvTramos.Columns.Add("col5", "Longitud");
        }
        public void Listar() { Buscar(); }
        public void Buscar()
        {
            dgvTramos.Rows.Clear();
            string texto = txtBuscar.Text.Trim().ToLower();
            string criterio = cboBuscarPor.Text;
            double desde = 0, hasta = 0;
            if (criterio == "Rango de kilómetros" && texto != "")
            {
                string[] partes = texto.Split(';');
                if (partes.Length != 2 || !double.TryParse(partes[0], out desde) ||
                    !double.TryParse(partes[1], out hasta) || double.IsNaN(desde) || double.IsNaN(hasta) ||
                    double.IsInfinity(desde) || double.IsInfinity(hasta) || desde < 0 || desde > hasta)
                {
                    lblTotal.Text = "Use desde;hasta, por ejemplo 0;20";
                    return;
                }
            }
            var datos = TListaTramo.GetLista().Where(op => texto == "" ||
                    (criterio == "Carretera" && (op.NombreCarretera.ToLower().Contains(texto) ||
op.Carretera.Codigo.ToString() == texto)) ||
                    (criterio == "Número" && op.Numero.ToString() == texto) ||
                    (criterio == "Rango de kilómetros" && op.KmInicial <= hasta && op.KmFinal >= desde));
            datos = datos.OrderBy(op => op.Carretera.Codigo).ThenBy(op => op.Numero);
            foreach (Tramo op in datos)
            {
                int fila = dgvTramos.Rows.Add(op.Carretera.Codigo, op.NombreCarretera, op.Numero, op.KmInicial,
op.KmFinal, op.Longitud);
                dgvTramos.Rows[fila].Tag = op;
            }
            lblTotal.Text = "Total de tramos: " + dgvTramos.Rows.Count;
            dgvTramos.ClearSelection();
        }
        private void Formulario_Load(object sender, EventArgs e) { CargarDatos(); Listar(); }
        private void Filtro_Cambiado(object sender, EventArgs e) { if (dgvTramos.Columns.Count > 0) Buscar(); }
        private void btnMostrarTodas_Click(object sender, EventArgs e) { txtBuscar.Clear();
cboBuscarPor.SelectedIndex = 0; Listar(); }
        private void btnRegresar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

Anexo A.9 — frmTramo

Archivo: Formularios/frmTramo.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmTramo : Form
    {
        public frmTramo()
        {
            InitializeComponent();
            CargarDatos();
            btnGuardar.Click += btnGuardar_Click;
            btnCancelar.Click += btnCancelar_Click;
            txtKmInicial.TextChanged += Kilometros_TextChanged;
            txtKmFinal.TextChanged += Kilometros_TextChanged;
            AcceptButton = btnGuardar;
            CancelButton = btnCancelar;
        }
        private bool modificando;
        public void CargarDatos()
        {
            cboCarretera.Items.Clear();
            cboCarretera.DropDownStyle = ComboBoxStyle.DropDownList;
            foreach (Carretera op in TListaCarretera.GetLista()) cboCarretera.Items.Add(op);
            txtLongitud.ReadOnly = true;
        }
        public Tramo CrearObjeto()
        {
            return new Tramo((Carretera)cboCarretera.SelectedItem,
                int.Parse(txtNumero.Text), double.Parse(txtKmInicial.Text), double.Parse(txtKmFinal.Text));
        }
        public bool ValidarDatos()
        {
            int numero;
            double inicio, final;
            if (cboCarretera.SelectedItem == null || !int.TryParse(txtNumero.Text, out numero) || numero <= 0 ||
                !double.TryParse(txtKmInicial.Text, out inicio) || !double.TryParse(txtKmFinal.Text, out final))
            {
                MessageBox.Show("Seleccione la carretera e ingrese número y kilómetros válidos.");
                return false;
            }
            Tramo op = CrearObjeto();
            if (!TListaTramo.Validar(op))
            {
                MessageBox.Show("El kilómetro inicial debe ser no negativo y el final debe ser mayor. Use números
 finitos.");
                return false;
            }
            if (!modificando && TListaTramo.Buscar(op.Carretera.Codigo, numero) != null)
            {
                MessageBox.Show("El número de tramo ya existe en esa carretera.");
                return false;
            }
            return true;
        }
        public void setDatos(Tramo op)
        {
            modificando = true;
            cboCarretera.SelectedItem = op.Carretera;
            cboCarretera.Enabled = false; // Un tramo no puede transferirse a otra carretera.
            txtNumero.Text = op.Numero.ToString();
            txtNumero.ReadOnly = true;
            txtKmInicial.Text = op.KmInicial.ToString();
            txtKmFinal.Text = op.KmFinal.ToString();
            CalcularLongitud();
        }
        public void CalcularLongitud()
        {
            double inicio, final;
            txtLongitud.Clear();
            if (double.TryParse(txtKmInicial.Text, out inicio) && double.TryParse(txtKmFinal.Text, out final) &&
                !double.IsNaN(final - inicio) && !double.IsInfinity(final - inicio) && final > inicio && inicio
>= 0)
                txtLongitud.Text = (final - inicio).ToString("0.##");
        }
        public void Guardar() { if (ValidarDatos()) DialogResult = DialogResult.OK; }
        private void btnGuardar_Click(object sender, EventArgs e) { Guardar(); }
        private void btnCancelar_Click(object sender, EventArgs e) { DialogResult = DialogResult.Cancel; }
        private void Kilometros_TextChanged(object sender, EventArgs e) { CalcularLongitud(); }
    }
}
```

Anexo A.10 — frmAdminComunaRecorrida

Archivo: Formularios/frmAdminComunaRecorrida.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmAdminComunaRecorrida : Form
    {
        public frmAdminComunaRecorrida()
        {
            InitializeComponent();
            Load += Formulario_Load;
            btnRegresar.Click += btnRegresar_Click;
            btnMostrarTodas.Click += btnMostrarTodas_Click;
            txtBuscar.TextChanged += Filtro_Cambiado;
            cboBuscarPor.SelectedIndexChanged += Filtro_Cambiado;
            btnInsertar.Click += btnInsertar_Click;
            btnModificar.Click += btnModificar_Click;
            btnEliminar.Click += btnEliminar_Click;
        }
        public void CargarDatos()
        {
            TListaDatos.CargarDatos();
            cboBuscarPor.Items.Clear();
            cboBuscarPor.Items.AddRange(new object[] { "Carretera", "Tramo", "Comuna", "Orden" });
            cboBuscarPor.DropDownStyle = ComboBoxStyle.DropDownList;
            cboBuscarPor.SelectedIndex = 0;
            dgvRecorridos.DataSource = null;
            dgvRecorridos.Columns.Clear();
            dgvRecorridos.AutoGenerateColumns = false;
            dgvRecorridos.ReadOnly = true;
            dgvRecorridos.AllowUserToAddRows = false;
            dgvRecorridos.AllowUserToDeleteRows = false;
            dgvRecorridos.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvRecorridos.MultiSelect = false;
            dgvRecorridos.Columns.Add("col0", "Código carretera");
            dgvRecorridos.Columns.Add("col1", "Carretera");
            dgvRecorridos.Columns.Add("col2", "Tramo");
            dgvRecorridos.Columns.Add("col3", "Comuna");
            dgvRecorridos.Columns.Add("col4", "Orden");
            dgvRecorridos.Columns.Add("col5", "Km entrada");
            dgvRecorridos.Columns.Add("col6", "Km salida");
        }
        public void Listar() { Buscar(); }
        public void Buscar()
        {
            dgvRecorridos.Rows.Clear();
            string texto = txtBuscar.Text.Trim().ToLower();
            string criterio = cboBuscarPor.Text;
            var datos = TListaComunaRecorrida.GetLista().Where(op => texto == "" ||
                    (criterio == "Carretera" && (op.NombreCarretera.ToLower().Contains(texto) ||
op.Tramo.Carretera.Codigo.ToString() == texto)) ||
                    (criterio == "Tramo" && op.NumeroTramo.ToString() == texto) ||
                    (criterio == "Comuna" && op.NombreComuna.ToLower().Contains(texto)) ||
                    (criterio == "Orden" && op.Orden.ToString() == texto));
            datos = datos.OrderBy(op => op.Tramo.Carretera.Codigo).ThenBy(op => op.NumeroTramo).ThenBy(op =>
op.Orden);
            foreach (ComunaRecorrida op in datos)
            {
                int fila = dgvRecorridos.Rows.Add(op.Tramo.Carretera.Codigo, op.NombreCarretera, op.NumeroTramo,
op.NombreComuna, op.Orden, op.KmEntrada, op.KmSalida);
                dgvRecorridos.Rows[fila].Tag = op;
            }
            lblTotal.Text = "Total de recorridos: " + dgvRecorridos.Rows.Count;
            dgvRecorridos.ClearSelection();
        }
        public void Nuevo()
        {
            if (TListaTramo.GetLista().Count == 0 || TListaComuna.GetLista().Count == 0)
            {
                MessageBox.Show("Se necesitan tramos y comunas registrados.");
                return;
            }
            using (frmComunaRecorrida frm = new frmComunaRecorrida())
            {
                if (frm.ShowDialog() == DialogResult.OK)
                {
                    bool resultado = TListaComunaRecorrida.Agregar(frm.CrearObjeto());
                    MessageBox.Show(resultado ? "Registro ingresado correctamente." : "No se pudo guardar; revise
 duplicados y relaciones.");
                    Listar();
                }
```

```
            }
        }
        public void Actualizar()
        {
            ComunaRecorrida op = Seleccionado();
            if (op == null) { MessageBox.Show("Seleccione una fila para modificar."); return; }
            using (frmComunaRecorrida frm = new frmComunaRecorrida())
            {
                frm.setDatos(op);
                if (frm.ShowDialog() == DialogResult.OK)
                {
                    bool resultado = TListaComunaRecorrida.Modificar(op, frm.CrearObjeto());
                    MessageBox.Show(resultado ? "Registro modificado correctamente." : "No se pudo modificar;
revise las relaciones existentes.");
                    Listar();
                }
            }
        }
        private void btnInsertar_Click(object sender, EventArgs e) { Nuevo(); }
        private void btnModificar_Click(object sender, EventArgs e) { Actualizar(); }
        private ComunaRecorrida Seleccionado()
        {
            if (dgvRecorridos.SelectedRows.Count == 0) return null;
            return dgvRecorridos.SelectedRows[0].Tag as ComunaRecorrida;
        }
        public void Eliminar()
        {
            ComunaRecorrida op = Seleccionado();
            if (op == null) { MessageBox.Show("Seleccione una fila para eliminar."); return; }
            if (MessageBox.Show("¿Desea eliminar el registro seleccionado?", "Eliminar", MessageBoxButtons.YesNo,
                MessageBoxIcon.Question) == DialogResult.Yes)
            {
                bool resultado = TListaComunaRecorrida.Eliminar(op.Tramo.Carretera.Codigo, op.Tramo.Numero,
op.Orden);
                MessageBox.Show(resultado ? "Registro eliminado correctamente." : "No se puede eliminar: tiene
relaciones con tramos o extremos.");
                Listar();
            }
        }
        private void btnEliminar_Click(object sender, EventArgs e) { Eliminar(); }
        private void Formulario_Load(object sender, EventArgs e) { CargarDatos(); Listar(); }
        private void Filtro_Cambiado(object sender, EventArgs e) { if (dgvRecorridos.Columns.Count > 0) Buscar();
 }
        private void btnMostrarTodas_Click(object sender, EventArgs e) { txtBuscar.Clear();
cboBuscarPor.SelectedIndex = 0; Listar(); }
        private void btnRegresar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

Anexo A.11 — frmComunaRecorrida

Archivo: Formularios/frmComunaRecorrida.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmComunaRecorrida : Form
    {
        public frmComunaRecorrida()
        {
            InitializeComponent();
            CargarDatos();
            label1.Text = "COMUNAS RECORRIDAS";
            label4.Text = "Kilómetro entrada:";
            label5.Text = "Kilómetro salida:";
            cboCarretera.SelectedIndexChanged += cboCarretera_SelectedIndexChanged;
            btnGuardar.Click += btnGuardar_Click;
            btnCancelar.Click += btnCancelar_Click;
            AcceptButton = btnGuardar;
            CancelButton = btnCancelar;
        }
        private ComunaRecorrida original;
        public void CargarDatos()
        {
            cboCarretera.Items.Clear();
            cboTramo.Items.Clear();
            cboComuna.Items.Clear();
            cboCarretera.DropDownStyle = cboTramo.DropDownStyle = cboComuna.DropDownStyle =
ComboBoxStyle.DropDownList;
            foreach (Carretera op in TListaCarretera.GetLista()) cboCarretera.Items.Add(op);
            foreach (Comuna op in TListaComuna.GetLista()) cboComuna.Items.Add(op);
        }
        public void CargarTramos()
        {
            cboTramo.Items.Clear();
            Carretera carretera = cboCarretera.SelectedItem as Carretera;
            if (carretera != null)
                foreach (Tramo op in TListaTramo.GetLista().Where(x => x.Carretera.Codigo ==
carretera.Codigo).OrderBy(x => x.Numero))
                    cboTramo.Items.Add(op);
        }
        public ComunaRecorrida CrearObjeto()
        {
            return new ComunaRecorrida((Tramo)cboTramo.SelectedItem, (Comuna)cboComuna.SelectedItem,
                int.Parse(txtOrden.Text), double.Parse(txtKmEntrada.Text), double.Parse(txtKmSalida.Text));
        }
        public bool ValidarDatos()
        {
            int orden;
            double entrada, salida;
            if (cboTramo.SelectedItem == null || cboComuna.SelectedItem == null ||
                !int.TryParse(txtOrden.Text, out orden) || orden <= 0 ||
                !double.TryParse(txtKmEntrada.Text, out entrada) || !double.TryParse(txtKmSalida.Text, out
salida))
            {
                MessageBox.Show("Seleccione carretera, tramo y comuna; ingrese orden positivo y kilómetros
válidos.");
                return false;
            }
            ComunaRecorrida op = CrearObjeto();
            if (!TListaComunaRecorrida.Validar(op))
            {
                MessageBox.Show("Entrada y salida deben estar dentro del tramo, con entrada menor o igual a
salida.");
                return false;
            }
            ComunaRecorrida existe = TListaComunaRecorrida.Buscar(op.Tramo.Carretera.Codigo, op.Tramo.Numero,
op.Orden);
            if (existe != null && existe != original)
            {
                MessageBox.Show("El orden ya está ocupado en ese tramo.");
                return false;
            }
            if (original != null && !TListaComunaRecorrida.PuedeModificar(original, op))
            {
                MessageBox.Show("El cambio dejaría un extremo fuera de su comuna recorrida. Modifique primero el
extremo.");
                return false;
            }
            return true;
        }
        public void setDatos(ComunaRecorrida op)
```

```
        {
            original = op;
            cboCarretera.SelectedItem = op.Tramo.Carretera;
            cboTramo.SelectedItem = op.Tramo;
            cboComuna.SelectedItem = op.Comuna;
            txtOrden.Text = op.Orden.ToString();
            txtKmEntrada.Text = op.KmEntrada.ToString();
            txtKmSalida.Text = op.KmSalida.ToString();
        }
        public void Guardar() { if (ValidarDatos()) DialogResult = DialogResult.OK; }
        private void cboCarretera_SelectedIndexChanged(object sender, EventArgs e) { CargarTramos(); }
        private void btnGuardar_Click(object sender, EventArgs e) { Guardar(); }
        private void btnCancelar_Click(object sender, EventArgs e) { DialogResult = DialogResult.Cancel; }
    }
}
```

Anexo A.12 — frmAdminExtremo

Archivo: Formularios/frmAdminExtremo.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmAdminExtremo : Form
    {
        public frmAdminExtremo()
        {
            InitializeComponent();
            Load += Formulario_Load;
            btnRegresar.Click += btnRegresar_Click;
            btnMostrarTodas.Click += btnMostrarTodas_Click;
            txtBuscar.TextChanged += Filtro_Cambiado;
            cboBuscarPor.SelectedIndexChanged += Filtro_Cambiado;
            btnInsertar.Click += btnInsertar_Click;
            btnModificar.Click += btnModificar_Click;
            btnEliminar.Click += btnEliminar_Click;
        }
        public void CargarDatos()
        {
            TListaDatos.CargarDatos();
            cboBuscarPor.Items.Clear();
            cboBuscarPor.Items.AddRange(new object[] { "Carretera", "Tipo de extremo", "Finalización", "Destino"
});
            cboBuscarPor.DropDownStyle = ComboBoxStyle.DropDownList;
            cboBuscarPor.SelectedIndex = 0;
            dgvExtremos.DataSource = null;
            dgvExtremos.Columns.Clear();
            dgvExtremos.AutoGenerateColumns = false;
            dgvExtremos.ReadOnly = true;
            dgvExtremos.AllowUserToAddRows = false;
            dgvExtremos.AllowUserToDeleteRows = false;
            dgvExtremos.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dgvExtremos.MultiSelect = false;
            dgvExtremos.Columns.Add("col0", "Código carretera");
            dgvExtremos.Columns.Add("col1", "Carretera");
            dgvExtremos.Columns.Add("col2", "Extremo");
            dgvExtremos.Columns.Add("col3", "Tramo");
            dgvExtremos.Columns.Add("col4", "Comuna");
            dgvExtremos.Columns.Add("col5", "Km");
            dgvExtremos.Columns.Add("col6", "Finalización");
            dgvExtremos.Columns.Add("col7", "Destino");
            dgvExtremos.Columns.Add("col8", "Tramo destino");
            dgvExtremos.Columns.Add("col9", "Comuna destino");
            dgvExtremos.Columns.Add("col10", "Km destino");
            dgvExtremos.Columns.Add("col11", "Observación");
        }
        public void Listar() { Buscar(); }
        public void Buscar()
        {
            dgvExtremos.Rows.Clear();
            string texto = txtBuscar.Text.Trim().ToLower();
            string criterio = cboBuscarPor.Text;
            var datos = TListaExtremoCarretera.GetLista().Where(op => texto == "" ||
                    (criterio == "Carretera" && (op.NombreCarretera.ToLower().Contains(texto) ||
op.Carretera.Codigo.ToString() == texto)) ||
                    (criterio == "Tipo de extremo" && op.TipoExtremo.ToLower().Contains(texto)) ||
                    (criterio == "Finalización" && op.TipoFinalizacion.ToLower().Contains(texto)) ||
                    (criterio == "Destino" && op.NombreCarreteraDestino.ToLower().Contains(texto)));
            foreach (ExtremoCarretera op in datos)
            {
                int fila = dgvExtremos.Rows.Add(op.Carretera.Codigo, op.NombreCarretera, op.TipoExtremo,
op.Tramo.Numero, op.NombreComuna, op.Kilometro, op.TipoFinalizacion, op.NombreCarreteraDestino, op.TramoDestino
== null ? "" : op.TramoDestino.Numero.ToString(), op.ComunaDestino == null ? "" : op.ComunaDestino.Nombre,
op.CarreteraDestino == null ? "" : op.KmDestino.ToString(), op.Observacion);
                dgvExtremos.Rows[fila].Tag = op;
            }
            lblTotal.Text = "Total de extremos: " + dgvExtremos.Rows.Count;
            dgvExtremos.ClearSelection();
        }
        public void Nuevo()
        {
            if (TListaTramo.GetLista().Count == 0 || TListaComuna.GetLista().Count == 0)
            {
                MessageBox.Show("Se necesitan tramos y comunas registrados.");
                return;
            }
            using (frmExtremo frm = new frmExtremo())
            {
                if (frm.ShowDialog() == DialogResult.OK)
```

```
                {
                    bool resultado = TListaExtremoCarretera.Agregar(frm.CrearObjeto());
                    MessageBox.Show(resultado ? "Registro ingresado correctamente." : "No se pudo guardar; revise
 duplicados y relaciones.");
                    Listar();
                }
            }
        }
        public void Actualizar()
        {
            ExtremoCarretera op = Seleccionado();
            if (op == null) { MessageBox.Show("Seleccione una fila para modificar."); return; }
            using (frmExtremo frm = new frmExtremo())
            {
                frm.setDatos(op);
                if (frm.ShowDialog() == DialogResult.OK)
                {
                    bool resultado = TListaExtremoCarretera.Modificar(frm.CrearObjeto());
                    MessageBox.Show(resultado ? "Registro modificado correctamente." : "No se pudo modificar;
revise las relaciones existentes.");
                    Listar();
                }
            }
        }
        private void btnInsertar_Click(object sender, EventArgs e) { Nuevo(); }
        private void btnModificar_Click(object sender, EventArgs e) { Actualizar(); }
        private ExtremoCarretera Seleccionado()
        {
            if (dgvExtremos.SelectedRows.Count == 0) return null;
            return dgvExtremos.SelectedRows[0].Tag as ExtremoCarretera;
        }
        public void Eliminar()
        {
            ExtremoCarretera op = Seleccionado();
            if (op == null) { MessageBox.Show("Seleccione una fila para eliminar."); return; }
            if (MessageBox.Show("¿Desea eliminar el registro seleccionado?", "Eliminar", MessageBoxButtons.YesNo,
                MessageBoxIcon.Question) == DialogResult.Yes)
            {
                bool resultado = TListaExtremoCarretera.Eliminar(op.Carretera.Codigo, op.TipoExtremo);
                MessageBox.Show(resultado ? "Registro eliminado correctamente." : "No se puede eliminar: tiene
relaciones con tramos o extremos.");
                Listar();
            }
        }
        private void btnEliminar_Click(object sender, EventArgs e) { Eliminar(); }
        private void Formulario_Load(object sender, EventArgs e) { CargarDatos(); Listar(); }
        private void Filtro_Cambiado(object sender, EventArgs e) { if (dgvExtremos.Columns.Count > 0) Buscar(); }
        private void btnMostrarTodas_Click(object sender, EventArgs e) { txtBuscar.Clear();
cboBuscarPor.SelectedIndex = 0; Listar(); }
        private void btnRegresar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

Anexo A.13 — frmExtremo

Archivo: Formularios/frmExtremo.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmExtremo : Form
    {
        public frmExtremo()
        {
            InitializeComponent();
            CargarDatos();
            label1.Text = "EXTREMO DE CARRETERA";
            label4.Text = "Kilómetro:";
            label10.Text = "Observación:";
            label8.Text = "Comuna destino:";
            cboCarretera.SelectedIndexChanged += cboCarretera_SelectedIndexChanged;
            cboCarreteraDestino.SelectedIndexChanged += cboCarreteraDestino_SelectedIndexChanged;
            cboTipoFinalizacion.SelectedIndexChanged += cboTipoFinalizacion_SelectedIndexChanged;
            btnGuardar.Click += btnGuardar_Click;
            btnCancelar.Click += btnCancelar_Click;
            AcceptButton = btnGuardar;
            CancelButton = btnCancelar;
            HabilitarDestino();
        }
        private bool modificando;
        public void CargarDatos()
        {
            foreach (ComboBox cbo in new ComboBox[] { cboCarretera, cboTramo, cboComuna, cboTipoExtremo,
                cboTipoFinalizacion, cboCarreteraDestino, cboTramoDestino, cboComunaDestino })
            {
                cbo.Items.Clear();
                cbo.DropDownStyle = ComboBoxStyle.DropDownList;
            }
            foreach (Carretera op in TListaCarretera.GetLista()) cboCarretera.Items.Add(op);
            foreach (Comuna op in TListaComuna.GetLista())
            {
                cboComuna.Items.Add(op);
                cboComunaDestino.Items.Add(op);
            }
            cboTipoExtremo.Items.AddRange(new object[] { "Inicio", "Final" });
            cboTipoFinalizacion.Items.AddRange(new object[] { "Finalización física", "Confluencia con otra
carretera" });
        }
        public void CargarTramos()
        {
            cboTramo.Items.Clear();
            cboCarreteraDestino.Items.Clear();
            Carretera carretera = cboCarretera.SelectedItem as Carretera;
            if (carretera != null)
            {
                foreach (Tramo op in TListaTramo.GetLista().Where(x => x.Carretera.Codigo ==
carretera.Codigo).OrderBy(x => x.Numero))
                    cboTramo.Items.Add(op);
                foreach (Carretera op in TListaCarretera.GetLista().Where(x => x.Codigo != carretera.Codigo))
                    cboCarreteraDestino.Items.Add(op);
            }
            CargarTramosDestino();
        }
        public void CargarTramosDestino()
        {
            cboTramoDestino.Items.Clear();
            Carretera carretera = cboCarreteraDestino.SelectedItem as Carretera;
            if (carretera != null)
                foreach (Tramo op in TListaTramo.GetLista().Where(x => x.Carretera.Codigo ==
carretera.Codigo).OrderBy(x => x.Numero))
                    cboTramoDestino.Items.Add(op);
        }
        public void HabilitarDestino()
        {
            bool confluencia = cboTipoFinalizacion.Text == "Confluencia con otra carretera";
            cboCarreteraDestino.Enabled = cboTramoDestino.Enabled = cboComunaDestino.Enabled =
                txtKmDestino.Enabled = txtObservacion.Enabled = confluencia;
            if (!confluencia)
            {
                cboCarreteraDestino.SelectedIndex = -1;
                cboTramoDestino.Items.Clear();
                cboComunaDestino.SelectedIndex = -1;
                txtKmDestino.Clear();
                txtObservacion.Clear();
            }
        }
```

```
        public ExtremoCarretera CrearObjeto()
        {
            bool confluencia = cboTipoFinalizacion.Text == "Confluencia con otra carretera";
            return new ExtremoCarretera((Carretera)cboCarretera.SelectedItem, cboTipoExtremo.Text,
                (Tramo)cboTramo.SelectedItem, (Comuna)cboComuna.SelectedItem, double.Parse(txtKilometro.Text),
                cboTipoFinalizacion.Text, confluencia ? (Carretera)cboCarreteraDestino.SelectedItem : null,
                confluencia ? (Tramo)cboTramoDestino.SelectedItem : null,
                confluencia ? (Comuna)cboComunaDestino.SelectedItem : null,
                confluencia ? double.Parse(txtKmDestino.Text) : 0, confluencia ? txtObservacion.Text.Trim() :
""");
        }
        public bool ValidarDatos()
        {
            double km;
            if (cboCarretera.SelectedItem == null || cboTramo.SelectedItem == null || cboComuna.SelectedItem ==
null ||
                cboTipoExtremo.SelectedItem == null || cboTipoFinalizacion.SelectedItem == null ||
                !double.TryParse(txtKilometro.Text, out km))
            {
                MessageBox.Show("Complete los datos del extremo e ingrese un kilómetro válido.");
                return false;
            }
            if (cboTipoFinalizacion.Text == "Confluencia con otra carretera" &&
                (cboCarreteraDestino.SelectedItem == null || cboTramoDestino.SelectedItem == null ||
                 cboComunaDestino.SelectedItem == null || !double.TryParse(txtKmDestino.Text, out km) ||
                 string.IsNullOrWhiteSpace(txtObservacion.Text)))
            {
                MessageBox.Show("Complete carretera, tramo, comuna, kilómetro de destino y observación.");
                return false;
            }
            ExtremoCarretera op = CrearObjeto();
            if (!TListaExtremoCarretera.Validar(op))
            {
                MessageBox.Show("Revise los rangos, la comuna recorrida y el inicio/final de la carretera. La
confluencia debe ser con otra carretera y dentro de su tramo y comuna.");
                return false;
            }
            if (!modificando && TListaExtremoCarretera.Buscar(op.Carretera.Codigo, op.TipoExtremo) != null)
            {
                MessageBox.Show("Esa carretera ya tiene registrado ese tipo de extremo.");
                return false;
            }
            return true;
        }
        public void setDatos(ExtremoCarretera op)
        {
            modificando = true;
            cboCarretera.SelectedItem = op.Carretera;
            cboTipoExtremo.SelectedItem = op.TipoExtremo;
            cboCarretera.Enabled = cboTipoExtremo.Enabled = false;
            cboTramo.SelectedItem = op.Tramo;
            cboComuna.SelectedItem = op.Comuna;
            txtKilometro.Text = op.Kilometro.ToString();
            cboTipoFinalizacion.SelectedItem = op.TipoFinalizacion;
            if (op.TipoFinalizacion == "Confluencia con otra carretera")
            {
                cboCarreteraDestino.SelectedItem = op.CarreteraDestino;
                cboTramoDestino.SelectedItem = op.TramoDestino;
                cboComunaDestino.SelectedItem = op.ComunaDestino;
                txtKmDestino.Text = op.KmDestino.ToString();
                txtObservacion.Text = op.Observacion;
            }
        }
        public void Guardar() { if (ValidarDatos()) DialogResult = DialogResult.OK; }
        private void cboCarretera_SelectedIndexChanged(object sender, EventArgs e) { CargarTramos(); }
        private void cboCarreteraDestino_SelectedIndexChanged(object sender, EventArgs e) {
CargarTramosDestino(); }
        private void cboTipoFinalizacion_SelectedIndexChanged(object sender, EventArgs e) { HabilitarDestino(); }
        private void btnGuardar_Click(object sender, EventArgs e) { Guardar(); }
        private void btnCancelar_Click(object sender, EventArgs e) { DialogResult = DialogResult.Cancel; }
    }
}
```

Anexo A.14 — frmConsultas

Archivo: Formularios/frmConsultas.cs

```
using SistemaCarreteras.Controladores;
using SistemaCarreteras.Entidades;
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;
namespace SistemaCarreteras.Formularios
{
    public partial class frmConsultas : Form
    {
        public frmConsultas()
        {
            InitializeComponent();
        }
        public void CargarDatos()
        {
            TListaDatos.CargarDatos();
            cboConsulta.Items.AddRange(new object[] {
                "1. Carreteras por categoría", "2. Tramos de una carretera",
                "3. Comunas recorridas por tramo", "4. Extremos de una carretera", "5. Confluencias entre
carreteras" });
            foreach (Categoria op in TListaCategoria.GetLista()) cboCategoria.Items.Add(op);
            foreach (Carretera op in TListaCarretera.GetLista()) cboCarretera.Items.Add(op);
            if (cboCategoria.Items.Count > 0) cboCategoria.SelectedIndex = 0;
            if (cboCarretera.Items.Count > 0) cboCarretera.SelectedIndex = 0;
            cboConsulta.SelectedIndex = 0;
        }
        public void CargarTramos()
        {
            cboTramo.Items.Clear();
            Carretera carretera = cboCarretera.SelectedItem as Carretera;
            if (carretera != null)
                foreach (Tramo op in TListaTramo.GetLista().Where(x => x.Carretera.Codigo ==
carretera.Codigo).OrderBy(x => x.Numero))
                    cboTramo.Items.Add(op);
            if (cboTramo.Items.Count > 0) cboTramo.SelectedIndex = 0;
        }
        public void HabilitarFiltros()
        {
            cboCategoria.Enabled = cboConsulta.SelectedIndex == 0;
            cboCarretera.Enabled = cboConsulta.SelectedIndex >= 1 && cboConsulta.SelectedIndex <= 3;
            cboTramo.Enabled = cboConsulta.SelectedIndex == 2;
            dgvConsulta.DataSource = null;
            lblTotal.Text = "Seleccione los filtros y pulse Consultar.";
        }
        public void Consultar()
        {
            Categoria categoria = cboCategoria.SelectedItem as Categoria;
            Carretera carretera = cboCarretera.SelectedItem as Carretera;
            Tramo tramo = cboTramo.SelectedItem as Tramo;
            dgvConsulta.DataSource = null;
            if ((cboConsulta.SelectedIndex == 0 && categoria == null) ||
                (cboConsulta.SelectedIndex >= 1 && cboConsulta.SelectedIndex <= 3 && carretera == null) ||
                (cboConsulta.SelectedIndex == 2 && tramo == null))
            {
                lblTotal.Text = "No hay datos para los filtros seleccionados.";
                return;
            }
            if (cboConsulta.SelectedIndex == 0)
            {
                var datos = TListaCarretera.GetLista().Where(x => x.Categoria.Codigo == categoria.Codigo)
                    .Select(x => new { x.Codigo, x.Nombre, Categoria = x.NombreCategoria,
                        LongitudTotal = TListaTramo.GetLista().Where(t => t.Carretera.Codigo == x.Codigo).Sum(t
=> t.Longitud) }).ToList();
                dgvConsulta.DataSource = datos;
            }
            else if (cboConsulta.SelectedIndex == 1)
            {
                var datos = TListaTramo.GetLista().Where(x => x.Carretera.Codigo == carretera.Codigo)
                    .OrderBy(x => x.Numero).Select(x => new { x.NombreCarretera, x.Numero, x.KmInicial,
x.KmFinal, x.Longitud }).ToList();
                dgvConsulta.DataSource = datos;
            }
            else if (cboConsulta.SelectedIndex == 2)
            {
                var datos = TListaComunaRecorrida.GetLista().Where(x => x.Tramo.Carretera.Codigo ==
carretera.Codigo && x.Tramo.Numero == tramo.Numero)
                    .OrderBy(x => x.Orden).Select(x => new { x.NombreCarretera, x.NumeroTramo, x.NombreComuna,
x.Orden, x.KmEntrada, x.KmSalida }).ToList();
                dgvConsulta.DataSource = datos;
            }
            else if (cboConsulta.SelectedIndex == 3)
            {
                var datos = TListaExtremoCarretera.GetLista().Where(x => x.Carretera.Codigo == carretera.Codigo)
```

```
                    .OrderByDescending(x => x.TipoExtremo).Select(x => new { x.NombreCarretera, x.TipoExtremo,
x.NombreTramo,
                        x.NombreComuna, x.Kilometro, x.TipoFinalizacion, x.NombreCarreteraDestino }).ToList();
                dgvConsulta.DataSource = datos;
            }
            else if (cboConsulta.SelectedIndex == 4)
            {
                var datos = TListaExtremoCarretera.GetLista().Where(x => x.TipoFinalizacion == "Confluencia con
otra carretera")
                    .Select(x => new { Origen = x.NombreCarretera, x.TipoExtremo, TramoOrigen = x.Tramo.Numero,
                        ComunaOrigen = x.NombreComuna, x.Kilometro, Destino = x.NombreCarreteraDestino,
                        TramoDestino = x.TramoDestino.Numero, ComunaDestino = x.ComunaDestino.Nombre,
x.KmDestino, x.Observacion }).ToList();
                dgvConsulta.DataSource = datos;
            }
            lblTotal.Text = "Total de resultados: " + dgvConsulta.Rows.Count;
        }
        private void frmConsultas_Load(object sender, EventArgs e) { CargarDatos(); }
        private void cboConsulta_SelectedIndexChanged(object sender, EventArgs e) { HabilitarFiltros(); }
        private void cboCarretera_SelectedIndexChanged(object sender, EventArgs e) { CargarTramos();
HabilitarFiltros(); }
        private void Filtro_Cambiado(object sender, EventArgs e) { dgvConsulta.DataSource = null; lblTotal.Text =
 "Pulse Consultar para actualizar."; }
        private void btnConsultar_Click(object sender, EventArgs e) { Consultar(); }
        private void btnRegresar_Click(object sender, EventArgs e) { Close(); }
    }
}
```

Anexo A.15 — Diseño de consultas

Archivo: Formularios/frmConsultas.Designer.cs

```
namespace SistemaCarreteras.Formularios
{
    partial class frmConsultas
    {
        private System.ComponentModel.IContainer components = null;
        protected override void Dispose(bool disposing)
        {
            if (disposing && components != null) components.Dispose();
            base.Dispose(disposing);
        }
        private void InitializeComponent()
        {
            this.cboConsulta = new System.Windows.Forms.ComboBox();
            this.cboCategoria = new System.Windows.Forms.ComboBox();
            this.cboCarretera = new System.Windows.Forms.ComboBox();
            this.cboTramo = new System.Windows.Forms.ComboBox();
            this.dgvConsulta = new System.Windows.Forms.DataGridView();
            this.btnConsultar = new System.Windows.Forms.Button();
            this.btnRegresar = new System.Windows.Forms.Button();
            this.lblTotal = new System.Windows.Forms.Label();
            this.lblConsulta = new System.Windows.Forms.Label();
            this.lblCategoria = new System.Windows.Forms.Label();
            this.lblCarretera = new System.Windows.Forms.Label();
            this.lblTramo = new System.Windows.Forms.Label();
            ((System.ComponentModel.ISupportInitialize)(this.dgvConsulta)).BeginInit();
            this.SuspendLayout();
            this.cboConsulta.SetBounds(130, 20, 470, 28);
            this.cboCategoria.SetBounds(130, 65, 200, 28);
            this.cboCarretera.SetBounds(130, 110, 470, 28);
            this.cboTramo.SetBounds(715, 110, 180, 28);
            this.cboConsulta.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cboCategoria.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cboCarretera.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cboTramo.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cboConsulta.SelectedIndexChanged += new
System.EventHandler(this.cboConsulta_SelectedIndexChanged);
            this.cboCarretera.SelectedIndexChanged += new
System.EventHandler(this.cboCarretera_SelectedIndexChanged);
            this.cboCategoria.SelectedIndexChanged += new System.EventHandler(this.Filtro_Cambiado);
            this.cboTramo.SelectedIndexChanged += new System.EventHandler(this.Filtro_Cambiado);
            this.lblConsulta.Text = "Consulta:";
            this.lblConsulta.SetBounds(20, 25, 105, 25);
            this.lblCategoria.Text = "Categoría:";
            this.lblCategoria.SetBounds(20, 70, 105, 25);
            this.lblCarretera.Text = "Carretera:";
            this.lblCarretera.SetBounds(20, 115, 105, 25);
            this.lblTramo.Text = "Tramo:";
            this.lblTramo.SetBounds(630, 115, 80, 25);
            this.btnConsultar.Text = "Consultar";
            this.btnConsultar.SetBounds(715, 20, 180, 35);
            this.btnConsultar.Click += new System.EventHandler(this.btnConsultar_Click);
            this.dgvConsulta.SetBounds(20, 165, 920, 340);
            this.dgvConsulta.ReadOnly = true;
            this.dgvConsulta.RowHeadersVisible = false;
            this.dgvConsulta.SelectionMode = System.Windows.Forms.DataGridViewSelectionMode.FullRowSelect;
            this.dgvConsulta.AllowUserToAddRows = false;
            this.dgvConsulta.AllowUserToDeleteRows = false;
            this.dgvConsulta.AutoSizeColumnsMode = System.Windows.Forms.DataGridViewAutoSizeColumnsMode.AllCells;
            this.dgvConsulta.Anchor = System.Windows.Forms.AnchorStyles.Top |
System.Windows.Forms.AnchorStyles.Bottom | System.Windows.Forms.AnchorStyles.Left |
System.Windows.Forms.AnchorStyles.Right;
            this.lblTotal.SetBounds(20, 525, 660, 30);
            this.lblTotal.Anchor = System.Windows.Forms.AnchorStyles.Bottom |
System.Windows.Forms.AnchorStyles.Left;
            this.btnRegresar.Text = "Regresar";
            this.btnRegresar.SetBounds(810, 520, 130, 35);
            this.btnRegresar.Anchor = System.Windows.Forms.AnchorStyles.Bottom |
System.Windows.Forms.AnchorStyles.Right;
            this.btnRegresar.Click += new System.EventHandler(this.btnRegresar_Click);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(960, 575);
            this.MinimumSize = new System.Drawing.Size(800, 580);
            this.Controls.AddRange(new System.Windows.Forms.Control[] { this.cboConsulta, this.cboCategoria,
this.cboCarretera,
                this.cboTramo, this.dgvConsulta, this.btnConsultar, this.btnRegresar, this.lblTotal,
                this.lblConsulta, this.lblCategoria, this.lblCarretera, this.lblTramo });
            this.Text = "Consultas LINQ de la red vial";
            this.StartPosition = System.Windows.Forms.FormStartPosition.CenterParent;
            this.Load += new System.EventHandler(this.frmConsultas_Load);
            ((System.ComponentModel.ISupportInitialize)(this.dgvConsulta)).EndInit();
            this.ResumeLayout(false);
        }
        private System.Windows.Forms.ComboBox cboConsulta;
        private System.Windows.Forms.ComboBox cboCategoria;
```

```
        private System.Windows.Forms.ComboBox cboCarretera;
        private System.Windows.Forms.ComboBox cboTramo;
        private System.Windows.Forms.DataGridView dgvConsulta;
        private System.Windows.Forms.Button btnConsultar;
        private System.Windows.Forms.Button btnRegresar;
        private System.Windows.Forms.Label lblTotal;
        private System.Windows.Forms.Label lblConsulta;
        private System.Windows.Forms.Label lblCategoria;
        private System.Windows.Forms.Label lblCarretera;
        private System.Windows.Forms.Label lblTramo;
    }
}
```

Anexo B.1 — TListaCategoria

Archivo: Controladores/TListaCategoria.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaCategoria
    {
        public static List<Categoria> listaCategorias = new List<Categoria>();
        public static bool Agregar(Categoria op)
        {
            bool resultado = false;
            if (op != null && op.Codigo > 0 &&
                !string.IsNullOrWhiteSpace(op.Nombre))
            {
                Categoria existe = Buscar(op.Codigo);
                if (existe == null)
                {
                    listaCategorias.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(Categoria op)
        {
            if (op == null || !(op.Codigo > 0 && !string.IsNullOrWhiteSpace(op.Nombre))) return false;
            Categoria actual = Buscar(op.Codigo);
            if (actual == null) return false;
            actual.Nombre = op.Nombre;
            return true;
        }
        public static bool Eliminar(int codigo)
        {
            bool resultado = false;
            Categoria op = Buscar(codigo);
            if (op != null)
            {
                bool utilizada = TListaCarretera.listaCarreteras.Any(
                    x => x.Categoria != null &&
                    x.Categoria.Codigo == codigo);
                if (!utilizada)
                {
                    listaCategorias.Remove(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static Categoria Buscar(int codigo)
        {
            Categoria resultado = listaCategorias.FirstOrDefault(
                x => x.Codigo == codigo);
            return resultado;
        }
        public static List<Categoria> GetLista()
        {
            return listaCategorias.ToList();
        }
    }
}
```

Anexo B.2 — TListaCarretera

Archivo: Controladores/TListaCarretera.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaCarretera
    {
        public static List<Carretera> listaCarreteras = new List<Carretera>();
        public static bool Agregar(Carretera op)
        {
            bool resultado = false;
            if (op != null && op.Codigo > 0 &&
                !string.IsNullOrWhiteSpace(op.Nombre) &&
                op.Categoria != null)
            {
                Carretera existe = Buscar(op.Codigo);
                bool categoriaExiste = TListaCategoria.listaCategorias.Any(
                    x => x.Codigo == op.Categoria.Codigo);
                if (existe == null && categoriaExiste)
                {
                    listaCarreteras.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(Carretera op)
        {
            if (op == null || !(op.Codigo > 0 && !string.IsNullOrWhiteSpace(op.Nombre))) return false;
            Carretera actual = Buscar(op.Codigo);
            if (actual == null) return false;
            if (op.Categoria == null || TListaCategoria.Buscar(op.Categoria.Codigo) == null) return false;
            actual.Nombre = op.Nombre;
            actual.Categoria = TListaCategoria.Buscar(op.Categoria.Codigo);
            return true;
        }
        public static bool Eliminar(int codigo)
        {
            bool resultado = false;
            Carretera op = Buscar(codigo);
            if (op != null)
            {
                bool tieneTramos = TListaTramo.listaTramos.Any(
                    x => x.Carretera != null &&
                    x.Carretera.Codigo == codigo);
                bool tieneExtremos = TListaExtremoCarretera.listaExtremos.Any(
                    x => (x.Carretera != null &&
                          x.Carretera.Codigo == codigo) ||
                         (x.CarreteraDestino != null &&
                          x.CarreteraDestino.Codigo == codigo));
                if (!tieneTramos && !tieneExtremos)
                {
                    listaCarreteras.Remove(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static Carretera Buscar(int codigo)
        {
            Carretera resultado = listaCarreteras.FirstOrDefault(
                x => x.Codigo == codigo);
            return resultado;
        }
        public static List<Carretera> GetLista()
        {
            return listaCarreteras.ToList();
        }
    }
}
```

Anexo B.3 — TListaComuna

Archivo: Controladores/TListaComuna.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaComuna
    {
        public static List<Comuna> listaComunas = new List<Comuna>();
        public static bool Agregar(Comuna op)
        {
            bool resultado = false;
            if (op != null && op.Codigo > 0 &&
                !string.IsNullOrWhiteSpace(op.Nombre) &&
                !string.IsNullOrWhiteSpace(op.Canton) &&
                !string.IsNullOrWhiteSpace(op.Provincia))
            {
                Comuna existe = Buscar(op.Codigo);
                if (existe == null)
                {
                    listaComunas.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(Comuna op)
        {
            if (op == null || !(op.Codigo > 0 && !string.IsNullOrWhiteSpace(op.Nombre) &&
!string.IsNullOrWhiteSpace(op.Canton) && !string.IsNullOrWhiteSpace(op.Provincia))) return false;
            Comuna actual = Buscar(op.Codigo);
            if (actual == null) return false;
            actual.Nombre = op.Nombre;
            actual.Canton = op.Canton;
            actual.Provincia = op.Provincia;
            return true;
        }
        public static bool Eliminar(int codigo)
        {
            bool resultado = false;
            Comuna op = Buscar(codigo);
            if (op != null)
            {
                bool tieneRecorridos = TListaComunaRecorrida.listaRecorridos.Any(
                    x => x.Comuna != null &&
                    x.Comuna.Codigo == codigo);
                bool tieneExtremos = TListaExtremoCarretera.listaExtremos.Any(
                    x => (x.Comuna != null &&
                          x.Comuna.Codigo == codigo) ||
                         (x.ComunaDestino != null &&
                          x.ComunaDestino.Codigo == codigo));
                if (!tieneRecorridos && !tieneExtremos)
                {
                    listaComunas.Remove(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static Comuna Buscar(int codigo)
        {
            Comuna resultado = listaComunas.FirstOrDefault(
                x => x.Codigo == codigo);
            return resultado;
        }
        public static List<Comuna> GetLista()
        {
            return listaComunas.ToList();
        }
    }
}
```

Anexo B.4 — TListaTramo

Archivo: Controladores/TListaTramo.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaTramo
    {
        public static List<Tramo> listaTramos = new List<Tramo>();
        public static bool Agregar(Tramo op)
        {
            bool resultado = false;
            if (Validar(op))
            {
                Tramo existe = Buscar(
                    op.Carretera.Codigo, op.Numero);
                if (existe == null && !TListaExtremoCarretera.GetLista().Any(x =>
                    x.Carretera.Codigo == op.Carretera.Codigo &&
                    ((x.TipoExtremo == "Inicio" && op.KmInicial < x.Kilometro) ||
                     (x.TipoExtremo == "Final" && op.KmFinal > x.Kilometro))))
                {
                    listaTramos.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(Tramo op)
        {
            if (!Validar(op)) return false;
            Tramo actual = Buscar(op.Carretera.Codigo, op.Numero);
            if (actual == null) return false;
            bool fuera = TListaComunaRecorrida.GetLista().Any(x =>
                x.Tramo.Carretera.Codigo == op.Carretera.Codigo && x.Tramo.Numero == op.Numero &&
                (x.KmEntrada < op.KmInicial || x.KmSalida > op.KmFinal));
            if (fuera) return false;
            // Evaluate proposed boundaries before mutating the canonical tramo.
            double minimo = listaTramos.Where(x => x.Carretera.Codigo == op.Carretera.Codigo)
                .Min(x => x == actual ? op.KmInicial : x.KmInicial);
            double maximo = listaTramos.Where(x => x.Carretera.Codigo == op.Carretera.Codigo)
                .Max(x => x == actual ? op.KmFinal : x.KmFinal);
            foreach (ExtremoCarretera x in TListaExtremoCarretera.GetLista())
            {
                if (x.Carretera.Codigo == op.Carretera.Codigo)
                {
                    if (x.TipoExtremo == "Inicio" && (x.Kilometro != minimo ||
                        (x.Tramo.Numero == op.Numero && x.Kilometro != op.KmInicial))) return false;
                    if (x.TipoExtremo == "Final" && (x.Kilometro != maximo ||
                        (x.Tramo.Numero == op.Numero && x.Kilometro != op.KmFinal))) return false;
                }
                if (x.TramoDestino != null && x.TramoDestino.Carretera.Codigo == op.Carretera.Codigo &&
                    x.TramoDestino.Numero == op.Numero && (x.KmDestino < op.KmInicial || x.KmDestino >
op.KmFinal)) return false;
            }
            actual.KmInicial = op.KmInicial;
            actual.KmFinal = op.KmFinal;
            return true;
        }
        public static bool Eliminar(int codigoCarretera, int numero)
        {
            bool resultado = false;
            Tramo op = Buscar(codigoCarretera, numero);
            if (op != null)
            {
                bool tieneRecorridos = TListaComunaRecorrida.listaRecorridos.Any(
                    x => x.Tramo != null &&
                    x.Tramo.Carretera != null &&
                    x.Tramo.Carretera.Codigo == codigoCarretera &&
                    x.Tramo.Numero == numero);
                bool tieneExtremos = TListaExtremoCarretera.listaExtremos.Any(
                    x => (x.Tramo != null &&
                          x.Tramo.Carretera != null &&
                          x.Tramo.Carretera.Codigo == codigoCarretera &&
                          x.Tramo.Numero == numero) ||
                         (x.TramoDestino != null &&
                          x.TramoDestino.Carretera != null &&
                          x.TramoDestino.Carretera.Codigo == codigoCarretera &&
                          x.TramoDestino.Numero == numero));
                if (!tieneRecorridos && !tieneExtremos)
                {
                    listaTramos.Remove(op);
                    resultado = true;
```

```
                }
            }
            return resultado;
        }
        public static Tramo Buscar(int codigoCarretera, int numero)
        {
            Tramo resultado = listaTramos.FirstOrDefault(
                x => x.Carretera != null &&
                x.Carretera.Codigo == codigoCarretera &&
                x.Numero == numero);
            return resultado;
        }
        public static bool Validar(Tramo op)
        {
            bool resultado = false;
            if (op != null && op.Carretera != null)
            {
                bool carreteraExiste = TListaCarretera.listaCarreteras.Any(
                    x => x.Codigo == op.Carretera.Codigo);
                if (carreteraExiste &&
                    op.Numero > 0 &&
                    op.KmInicial >= 0 &&
                    op.KmFinal > op.KmInicial &&
                    !double.IsNaN(op.KmInicial) &&
                    !double.IsNaN(op.KmFinal) &&
                    !double.IsInfinity(op.KmInicial) &&
                    !double.IsInfinity(op.KmFinal) &&
                    !double.IsInfinity(op.KmFinal - op.KmInicial))
                {
                    resultado = true;
                }
            }
            return resultado;
        }
        public static List<Tramo> GetLista()
        {
            return listaTramos.ToList();
        }
    }
}
```

Anexo B.5 — TListaComunaRecorrida

Archivo: Controladores/TListaComunaRecorrida.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaComunaRecorrida
    {
        public static List<ComunaRecorrida> listaRecorridos =
            new List<ComunaRecorrida>();
        public static bool Agregar(ComunaRecorrida op)
        {
            bool resultado = false;
            if (Validar(op))
            {
                ComunaRecorrida existe = Buscar(
                    op.Tramo.Carretera.Codigo,
                    op.Tramo.Numero,
                    op.Orden);
                if (existe == null)
                {
                    listaRecorridos.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(ComunaRecorrida op)
        {
            if (op == null || op.Tramo == null || op.Tramo.Carretera == null) return false;
            return Modificar(Buscar(op.Tramo.Carretera.Codigo, op.Tramo.Numero, op.Orden), op);
        }
        public static bool PuedeModificar(ComunaRecorrida original, ComunaRecorrida nuevo)
        {
            if (original == null || !listaRecorridos.Contains(original) || !Validar(nuevo)) return false;
            ComunaRecorrida existe = Buscar(nuevo.Tramo.Carretera.Codigo, nuevo.Tramo.Numero, nuevo.Orden);
            if (existe != null && existe != original) return false;
            var restantes = listaRecorridos.Where(x => x != original).Concat(new[] { nuevo });
            return ConservaExtremos(restantes);
        }
        private static bool ConservaExtremos(IEnumerable<ComunaRecorrida> recorridos)
        {
            foreach (ExtremoCarretera extremo in TListaExtremoCarretera.GetLista())
            {
                if (!ContienePunto(recorridos, extremo.Tramo, extremo.Comuna, extremo.Kilometro)) return false;
                if (extremo.TipoFinalizacion == "Confluencia con otra carretera" &&
                    !ContienePunto(recorridos, extremo.TramoDestino, extremo.ComunaDestino, extremo.KmDestino))
return false;
            }
            return true;
        }
        public static bool ContienePunto(IEnumerable<ComunaRecorrida> recorridos, Tramo tramo, Comuna comuna,
double km)
        {
            if (tramo == null || tramo.Carretera == null || comuna == null) return false;
            return recorridos.Any(x => x.Tramo.Carretera.Codigo == tramo.Carretera.Codigo &&
                x.Tramo.Numero == tramo.Numero && x.Comuna.Codigo == comuna.Codigo &&
                km >= x.KmEntrada && km <= x.KmSalida);
        }
        public static bool Modificar(ComunaRecorrida original, ComunaRecorrida nuevo)
        {
            if (!PuedeModificar(original, nuevo)) return false;
            int pos = listaRecorridos.IndexOf(original);
            listaRecorridos[pos] = nuevo;
            return true;
        }
        public static bool Eliminar(int codigoCarretera,
            int numeroTramo, int orden)
        {
            bool resultado = false;
            ComunaRecorrida op = Buscar(
                codigoCarretera, numeroTramo, orden);
            if (op != null && ConservaExtremos(listaRecorridos.Where(x => x != op)))
            {
                listaRecorridos.Remove(op);
                resultado = true;
            }
            return resultado;
        }
        public static ComunaRecorrida Buscar(
            int codigoCarretera, int numeroTramo, int orden)
        {
```

```
            ComunaRecorrida resultado = listaRecorridos.FirstOrDefault(
                x => x.Tramo != null &&
                x.Tramo.Carretera != null &&
                x.Tramo.Carretera.Codigo == codigoCarretera &&
                x.Tramo.Numero == numeroTramo &&
                x.Orden == orden);
            return resultado;
        }
        public static bool Validar(ComunaRecorrida op)
        {
            bool resultado = false;
            if (op != null && op.Tramo != null &&
                op.Tramo.Carretera != null && op.Comuna != null)
            {
                Tramo tramo = TListaTramo.Buscar(
                    op.Tramo.Carretera.Codigo,
                    op.Tramo.Numero);
                Comuna comuna = TListaComuna.Buscar(op.Comuna.Codigo);
                if (tramo != null && comuna != null &&
                    op.Orden > 0 &&
                    !double.IsNaN(op.KmEntrada) &&
                    !double.IsNaN(op.KmSalida) &&
                    !double.IsInfinity(op.KmEntrada) &&
                    !double.IsInfinity(op.KmSalida) &&
                    op.KmEntrada >= tramo.KmInicial &&
                    op.KmSalida <= tramo.KmFinal &&
                    op.KmEntrada <= op.KmSalida)
                {
                    resultado = true;
                }
            }
            return resultado;
        }
        public static List<ComunaRecorrida> GetLista()
        {
            return listaRecorridos.ToList();
        }
    }
}
```

Anexo B.6 — TListaExtremoCarretera

Archivo: Controladores/TListaExtremoCarretera.cs

```
using SistemaCarreteras.Entidades;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SistemaCarreteras.Controladores
{
    public class TListaExtremoCarretera
    {
        public static List<ExtremoCarretera> listaExtremos =
            new List<ExtremoCarretera>();
        public static bool Agregar(ExtremoCarretera op)
        {
            bool resultado = false;
            if (Validar(op))
            {
                ExtremoCarretera existe = Buscar(
                    op.Carretera.Codigo, op.TipoExtremo);
                if (existe == null)
                {
                    listaExtremos.Add(op);
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Modificar(ExtremoCarretera op)
        {
            bool resultado = false;
            if (Validar(op))
            {
                int pos = listaExtremos.FindIndex(
                    x => x.Carretera != null &&
                    x.Carretera.Codigo == op.Carretera.Codigo &&
                    string.Equals(x.TipoExtremo, op.TipoExtremo,
                        StringComparison.OrdinalIgnoreCase));
                if (pos >= 0)
                {
                    listaExtremos[pos] = op;
                    resultado = true;
                }
            }
            return resultado;
        }
        public static bool Eliminar(int codigoCarretera,
            string tipoExtremo)
        {
            bool resultado = false;
            ExtremoCarretera op = Buscar(
                codigoCarretera, tipoExtremo);
            if (op != null)
            {
                listaExtremos.Remove(op);
                resultado = true;
            }
            return resultado;
        }
        public static ExtremoCarretera Buscar(
            int codigoCarretera, string tipoExtremo)
        {
            ExtremoCarretera resultado = listaExtremos.FirstOrDefault(
                x => x.Carretera != null &&
                x.Carretera.Codigo == codigoCarretera &&
                string.Equals(x.TipoExtremo, tipoExtremo,
                    StringComparison.OrdinalIgnoreCase));
            return resultado;
        }
        public static bool Validar(ExtremoCarretera op)
        {
            bool resultado = false;
            if (op != null && op.Carretera != null &&
                op.Tramo != null && op.Comuna != null)
            {
                bool tipoValido =
                    op.TipoExtremo == "Inicio" ||
                    op.TipoExtremo == "Final";
                bool finalizacionValida =
                    op.TipoFinalizacion == "Finalización física" ||
                    op.TipoFinalizacion == "Confluencia con otra carretera";
                Carretera carretera = TListaCarretera.Buscar(
                    op.Carretera.Codigo);
                Tramo tramo = TListaTramo.Buscar(
                    op.Carretera.Codigo, op.Tramo.Numero);
```

```
                Comuna comuna = TListaComuna.Buscar(op.Comuna.Codigo);
                if (tipoValido && finalizacionValida &&
                    carretera != null && tramo != null &&
                    comuna != null &&
                    op.Tramo.Carretera != null && op.Tramo.Carretera.Codigo == carretera.Codigo &&
                    EsLimite(op, tramo) &&
                    TListaComunaRecorrida.ContienePunto(TListaComunaRecorrida.GetLista(), tramo, comuna,
op.Kilometro) &&
                    !double.IsNaN(op.Kilometro) &&
                    !double.IsInfinity(op.Kilometro) &&
                    op.Kilometro >= tramo.KmInicial &&
                    op.Kilometro <= tramo.KmFinal)
                {
                    if (op.TipoFinalizacion == "Finalización física")
                    {
                        resultado = true;
                    }
                    else
                    {
                        if (op.CarreteraDestino != null &&
                            op.TramoDestino != null &&
                            op.ComunaDestino != null)
                        {
                            Carretera destino = TListaCarretera.Buscar(
                                op.CarreteraDestino.Codigo);
                            Tramo tramoDestino = TListaTramo.Buscar(
                                op.CarreteraDestino.Codigo,
                                op.TramoDestino.Numero);
                            Comuna comunaDestino = TListaComuna.Buscar(
                                op.ComunaDestino.Codigo);
                            if (destino != null &&
                                tramoDestino != null &&
                                comunaDestino != null &&
                                op.TramoDestino.Carretera != null && op.TramoDestino.Carretera.Codigo ==
destino.Codigo &&
                                !string.IsNullOrWhiteSpace(op.Observacion) &&
                                TListaComunaRecorrida.ContienePunto(TListaComunaRecorrida.GetLista(),
tramoDestino, comunaDestino, op.KmDestino) &&
                                destino.Codigo != carretera.Codigo &&
                                !double.IsNaN(op.KmDestino) &&
                                !double.IsInfinity(op.KmDestino) &&
                                op.KmDestino >= tramoDestino.KmInicial &&
                                op.KmDestino <= tramoDestino.KmFinal)
                            {
                                resultado = true;
                            }
                        }
                    }
                }
            }
            return resultado;
        }
        private static bool EsLimite(ExtremoCarretera op, Tramo tramo)
        {
            var tramos = TListaTramo.GetLista().Where(x => x.Carretera.Codigo == op.Carretera.Codigo).ToList();
            if (tramos.Count == 0) return false;
            if (op.TipoExtremo == "Inicio")
                return op.Kilometro == tramo.KmInicial && op.Kilometro == tramos.Min(x => x.KmInicial);
            return op.Kilometro == tramo.KmFinal && op.Kilometro == tramos.Max(x => x.KmFinal);
        }
        public static List<ExtremoCarretera> GetLista()
        {
            return listaExtremos.ToList();
        }
    }
}
```

Anexo B.7 — TListaDatos

Archivo: Controladores/TListaDatos.cs

```
using SistemaCarreteras.Entidades;
using System;
namespace SistemaCarreteras.Controladores
{
    // Datos de práctica; se cargan una sola vez por ejecución, nunca al volver a una pantalla.
    public class TListaDatos
    {
        private static bool cargado;
        public static void CargarDatos()
        {
            if (cargado) return;
            cargado = true;
            if (TListaCategoria.GetLista().Count != 0 || TListaCarretera.GetLista().Count != 0 ||
                TListaComuna.GetLista().Count != 0 || TListaTramo.GetLista().Count != 0 ||
                TListaComunaRecorrida.GetLista().Count != 0 || TListaExtremoCarretera.GetLista().Count != 0)
return;
            TListaCategoria.Agregar(new Categoria(1, "Local"));
            TListaCategoria.Agregar(new Categoria(2, "Comercial"));
            TListaCategoria.Agregar(new Categoria(3, "Regional"));
            TListaCategoria.Agregar(new Categoria(4, "Nacional"));
            TListaCategoria.Agregar(new Categoria(5, "Autovía"));
            TListaCarretera.Agregar(new Carretera(101, "Vía Machala - Pasaje", TListaCategoria.Buscar(3)));
            TListaCarretera.Agregar(new Carretera(102, "Vía Machala - Santa Rosa", TListaCategoria.Buscar(4)));
            TListaCarretera.Agregar(new Carretera(103, "Vía Puerto Bolívar", TListaCategoria.Buscar(1)));
            TListaCarretera.Agregar(new Carretera(104, "Vía Comercial Central", TListaCategoria.Buscar(2)));
            TListaCarretera.Agregar(new Carretera(105, "Autovía Principal", TListaCategoria.Buscar(5)));
            TListaComuna.Agregar(new Comuna(1, "Machala", "Machala", "El Oro"));
            TListaComuna.Agregar(new Comuna(2, "El Cambio", "Machala", "El Oro"));
            TListaComuna.Agregar(new Comuna(3, "Pasaje", "Pasaje", "El Oro"));
            TListaComuna.Agregar(new Comuna(4, "Santa Rosa", "Santa Rosa", "El Oro"));
            TListaComuna.Agregar(new Comuna(5, "Puerto Bolívar", "Machala", "El Oro"));
            TListaComuna.Agregar(new Comuna(6, "Comuna sin relaciones", "Arenillas", "El Oro"));
            foreach (Carretera carretera in TListaCarretera.GetLista())
            {
                TListaTramo.Agregar(new Tramo(carretera, 1, 0, 10));
                TListaTramo.Agregar(new Tramo(carretera, 2, 10, 20));
                Tramo primero = TListaTramo.Buscar(carretera.Codigo, 1);
                Tramo segundo = TListaTramo.Buscar(carretera.Codigo, 2);
                TListaComunaRecorrida.Agregar(new ComunaRecorrida(primero, TListaComuna.Buscar(1), 1, 0, 5));
                TListaComunaRecorrida.Agregar(new ComunaRecorrida(primero, TListaComuna.Buscar(2), 2, 5, 10));
                int codigoComuna = carretera.Codigo == 102 ? 4 : carretera.Codigo == 103 ? 5 : 3;
                TListaComunaRecorrida.Agregar(new ComunaRecorrida(segundo, TListaComuna.Buscar(codigoComuna), 1,
10, 20));
                TListaExtremoCarretera.Agregar(new ExtremoCarretera(carretera, "Inicio", primero,
                    TListaComuna.Buscar(1), 0, "Finalización física", null, null, null, 0, ""));
            }
            // Ejemplo ficticio de confluencia en El Cambio (no representa cartografía real).
            TListaExtremoCarretera.Agregar(new ExtremoCarretera(TListaCarretera.Buscar(101), "Final",
                TListaTramo.Buscar(101, 2), TListaComuna.Buscar(3), 20, "Confluencia con otra carretera",
                TListaCarretera.Buscar(102), TListaTramo.Buscar(102, 1), TListaComuna.Buscar(2), 7,
                "Ejemplo académico de unión vial"));
        }
    }
}
```

Anexo B.8 — Punto de entrada

Archivo: Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace SistemaCarreteras
{
    internal static class Program
    {
        /// <summary>
        /// Punto de entrada principal para la aplicación.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Formularios.frmPrincipal());
        }
    }
}
```